

cvc5's rewrite holes, reproved and elaborated by Carcara: what the pipeline is made of, and what run **rw5** costs

Carcara branch `egglog/rewrite-holes` (= `egglog/bounded-parallel-holes`)

Run **rw5** of 2026-09-27/28 (Carcara `7ec65b7d`) and the **dsl1** arm of 2026-09-25 – report of 2026-09-29

Abstract

At `--proof-granularity=rewrite`, **cvc5** prints one premise-free **hole** step per call of its rewriter. Carcara reproves each hole with a normalizer built from its own checker rules or with **egglog** saturation over **RARE** rules, reconstructs an Alethe derivation, and splices it in. This report describes, briefly, the parts added to that pipeline in the last weeks (§2), and answers four questions from run **rw5**: 7,381 benchmarks of **QF_UF**, **QF_LIA** and **QF_LRA**, 2,593,653 distinct holes in the 7,159 proofs whose elaboration pass finished (§3). **(1)** The normalizer closes 40.0% of the holes at a few milliseconds each and gives **egglog** a normal form for another 28.8%; measured on and off before **rw5**, it is the difference between 71% and 98% of the holes proved. **(2)** A hole **egglog** proves costs 0.2 s (**QF_UF**), 0.5 s (**QF_LRA**) or 1.2 s (**QF_LIA**) of worker time on average. **(3)** The reconstruction is 11% of that time and 12% of the pass; the elaborated proofs re-check 25 times faster than they are produced. **(4)** **cvc5**'s own expansion (**dsl1-rewrite**) gives 5,245 valid proofs against 4,679, in a seventh of the time.

1 Setting

Each benchmark runs on half an **octa** node (8 cores, 60 GB):

1. **cvc5** (branch `alethe-rewrite-granularity`) with `--proof-granularity=rewrite --proof-alethe-res-pivots --proof-elim-subtypes --print-arith-lit-token, 60s`. Its holes are tagged `MACRO_REWRITE` or `MACRO_SR_PRED_INTRO`; the arithmetic preprocessing and subtype-elimination trust steps are attempted too; the theory lemmas (`THEORY_LEMMA`, `DIAMONDS`) never.
2. Pass 0, `carcara elaborate --pipeline hoist prune, 300s` (§2.1).
3. Pass 1, `carcara elaborate --pipeline hole` with a 92-rule **RARE** file: the normalizer, then the holes on 8 workers, 60s and 5.5 GB per hole, 1,500s for the pass; the certificates are checked as they are spliced in.
4. `carcara check` of the elaborated proof, 1,200s.

The comparison arm **dsl1** runs the same **cvc5** at `--proof-granularity=dsl1-rewrite`, where **cvc5** expands every rewrite into **RARE** rule applications itself, and checks the proof with `carcara check` (900s); no **egglog**.

2 The pipeline's parts

2.1 Before any hole: hoist and prune

cvc5 prints a rewrite once for every subproof that uses it. Pass 0 (`6364ff71`) checks the proof's steps, computes a digest of every closed derivation (holes included), lifts each one that repeats

to the top level once and makes its copies references; **prune** then drops what the conclusion no longer reaches. The hoisted proof is printed with term sharing: printed unshared, a 2.4 MB proof had become 12 GB. Within the pass, holes with the same goal under the same assumptions share one run (the verdict memo). Table 1: 4.6 M hole steps become 2.6 M distinct holes, 2.3–2.4 times fewer on the arithmetic logics, in 3.1 h summed; 206 proofs are rejected before any hole by **cvc5**’s resolution-pivot defect.

	QF_UF	QF_LIA	QF_LRA	all
proofs (complete in 60 s)	4,316	2,539	524	7,379
hoisted	4,118	2,529	518	7,165
rejected upfront (cvc5 ’s resolution pivots)	198	2	6	206
hoist out of time (300 s)	0	8	0	8
hoisted proofs: hole steps printed by cvc5	1,239,039	2,132,077	1,261,973	4,633,089
distinct holes after hoist and prune	1,171,831	915,032	522,311	2,609,174
ratio	1.06	2.33	2.42	1.78
proofs with fewer holes after it	2,271	1,231	386	3,888
commands pruned, of all	0.3%	20.5%	1.1%	2.7%
time, median / max	363 ms / 169.7 s	14 ms / 238.3 s	74 ms / 20.0 s	168 ms / 238.3 s
time, summed (h)	1.3	1.6	0.1	3.1

Table 1: Pass 0 on the proofs **cvc5** completed in 60 s.

2.2 The pre-normalizer

`--hole-prenormalize (src/elaborator/prenorm.rs)`. Before any hole is scheduled, both sides of every hole are normalized bottom-up, with one memo per proof, by five of the checker’s own procedures: **evaluate** on ground terms; **poly_simp**, arithmetic to the checker’s canonical polynomial; **poly_simp_rel**, a relation to `(op P c)` with the difference scaled to integral coefficients of gcd 1 (Int) or leading coefficient 1 (Real), equalities oriented; **aci_simp**, and/or flattened, identities dropped, duplicates removed, arguments sorted; **distinct_elim**; and one fold, two mirrored bounds among a conjunction’s arguments to their equality (**la_rw_eq**). Every step is one rule applied to one subterm, so a normal form comes with its certificate (**cong**, **trans**, **symm** around those steps). When the two sides meet, that certificate replaces the hole and **egglog** never runs. Otherwise **egglog** gets the equality of the normal forms first, on a third of the hole’s budget, and its certificate is bridged back to the stated sides by the normalizer’s derivations; if that fails, the goal as stated gets the whole budget. The normalizer stays within core Alethe: Boolean simplification, relation flips and integer tightening are RARE rules, and **egglog**’s job. Its orders are term pointers, canonical within a run but not between runs, which is where **rw5**’s run-to-run differences on **rings** came from.

2.3 The set encoding of and/or

egglog stores a term’s arguments as a chain, and a RARE rule with k `:list` parameters has to bind segments of it: the **chain** encoding does that by re-association and $2^k - 1$ variants of the rule for the empty lists. `--rare-list-encoding set-form` (the default) also represents every **and/or** as the set of its arguments, with rules for identity and absorbing elements, and compiles a rule whose left-hand side is an **and/or** over `:list` parameters *once* on that form, its fixed arguments becoming membership conditions: it fires at any arity, in any position, with empty lists. The set forms of derived terms come from a ruleset of their own that runs only as **egglog**’s last fallback, so a goal the ordinary rules prove pays nothing for them; since **rw5**, membership is a relation filled by position and splicing a nested **and/or** into its parent is guarded. Measured before: on **veriT**’s Boolean-heavy holes the set form proves 83% more holes in the same budget (98 proofs). On **cvc5**’s holes the two encodings are a wash: over the whole corpus at theory-rewrite granularity (**enc4**)

they prove 70.7% and 70.6% of the holes without the normalizer and 98.3% each with it, in 112h and 113h, and 50h each.

2.4 Sort guards

egglog’s rules range over one untyped sort of terms, so an arithmetic rule matched Boolean and uninterpreted terms: `arith-eq-elim` fired 12,000 times on `QF_UF` goals. `--rare-sort-guards` (58bc6945) keeps relations `SortInt`, `SortReal`, `SortBool` (later `String`, `RegLan`) on e-classes, seeded from the goal’s subterms by their declared sorts and propagated by operator (`Bool` for connectives and comparisons, `Real` for `/` and `to_real`, the argument’s sort for `+` `-` `*`, the branch’s for `ite`, the declared result for an uninterpreted function), and adds a guard premise for every `Int`, `Real` or `Bool` parameter of a rule. The certificate search strips the guards, so the certificates do not change. On the ten sample proofs of the time the kept holes went from 267 to 169 with the ACI identity rules of the same day, and to 129 with the guards; micro-goals ran 2 to 4 times faster.

2.5 Premise seeding

A conditional RARE rule fires only when its premise holds in the e-graph, so the premise’s terms must exist, and the rule compiler adds a *demand* rule that builds them. The first demand built every premise over all pairs of available terms, whatever their sorts: on a `QF_UF` goal of twelve conjuncts, eight such rules fired 2,175 times each in the second round and 12,775 in the fourth, and `eq-cond-deq` matched 530,000 times in one round; a 28-conjunct hole of the corpus died at 6 GB, where its proof had checked in under a second at theory-rewrite granularity. Two designs were measured. Seeding the premises from the goal’s subterms only (`--rare-seed-from-goal`, 29e4f7f3) was 5% faster and lost 9 provable holes of the sample, so it is not used. *Demand at the left-hand side* (d78da84d, with 6fa30dbf) builds a premise’s terms exactly for the bindings of each occurrence of the rule’s left-hand side, where the rule could fire; a premise variable the left-hand side does not bind ranges over the seed relation under its sort guard, and no all-pairs seeding is left. Such goals then take 0.2–0.3s at every size up to 28 conjuncts; on the ten samples every hole but two is proved and the passes run 1.3 to 10 times faster; on 23 local proofs the unproved holes went from 56 to 6 and the pass from 903s to 475s. `rw2`, the first run with it (and with a larger pass budget), checked 99.9/99.7/99.5% of the holes (`QF_UF/QF_LIA/QF_LRA`) against `rw1`’s 99.6/95.8/85.7%.

2.6 The parallel setup

- **A process per hole** (`--hole-isolate`), eight at a time (`--hole-threads 8`), under a hard time limit (the child is killed at 60s) and an address-space limit of 5.5 GB, the job’s 60 GB less 16 GB for the driver over eight, with a soft cap at 90% for a clean stop. A killed or failed child leaves its hole as a trusted hole with a reason class, and the rest of the proof goes on. A process is the only hard bound: an egglog iteration cannot be interrupted, and in one process a 1s budget was overrun sixty-fold.
- **A process per attempt.** Within a hole every attempt runs in a fresh process with its share of the hole’s budget: the normalized goal (a third), the abstract goal (shared subterms of 16 nodes or more as constants; half), the whole goal (a quarter, at most 10s), the structural descent over atom pairs (at least 8s a pair), the atom alignment, the whole goal again. Each is killed 3s past its share and dies with its parent. A process that had failed once was where every later attempt met the memory limit.
- **A pass budget** of 1,500s from Carcara’s start: holes not started when it runs out are kept, and the pass ends with its partial result. Holes are scheduled smallest goal first, so a binding budget covers the most holes (12 to 75 times more on three `QF_LRA` proofs).

- **Text across the boundary.** A child returns the certificate as text, which the parent parses, checks and splices into the proof, one hole at a time.
- **A runner whose records survive a kill:** each pass’s keys are printed as soon as they are known, and the kept holes are counted by class.

In **rw5** (Table 6) the workers are busy 73% of the hole phase (QF_UF 56%: small proofs and the tails of large ones), starting a process and parsing its input costs 25 ms a hole (10% of the worker time), no task used more than 37.8 GB of its 60 GB, the pass budget cut 36 holes, and every task kept its record. 14 passes (Dartagnan and **nec-smt** proofs) were killed at the external limit of 1,600 s and count as unfinished.

3 Four questions

3.1 With the normalizer, or without?

rw5 ran with the normalizer, so its effect is read within the run (Table 2) and from two earlier on/off measurements (Table 3).

Within rw5. The normalizer closes 1,036,355 holes, 40.0% (QF_UF 13%, QF_LIA 61%, QF_LRA 64%), for 466 s of normalization and 0.75 h of certificate checking in all: 3 ms a hole, against 0.2 to 1.2 s for a hole egglog proves. It gives egglog a normal form for another 28.8% of the holes, and 99.24% of those are proved and bridged; of the 5,660 that are not, 3,997 are proved as stated. The normal forms are smaller than the goals for 86% of the QF_UF ones and larger for 52% of QF_LIA’s (the polynomial form with its $(\ast -1 \ x)$ monomials). The remaining 31% of the holes are goals the normalizer leaves unchanged, and none of the 1,653 kept holes is among them: every kept hole had been given a normal form first. Sent to egglog instead, the closed holes would cost at least the floor of a worker hole (the cheapest 1% cost 0.15 s in QF_UF, 0.16 s in QF_LIA, 0.21 s in QF_LRA): about 50 h of worker time more, nearly a quarter on top of the run’s 216.8 h.

	QF_UF	QF_LIA	QF_LRA	all
holes (proofs whose pass finished)	1,171,831	899,511	522,311	2,593,653
closed by the normalizer	155,455 (13.3%)	549,155 (61.1%)	331,745 (63.5%)	1,036,355 (40.0%)
normal form tried first	340,595 (29.1%)	245,973 (27.3%)	160,856 (30.8%)	747,424 (28.8%)
proved and bridged	339,758 (99.75%)	241,402 (98.14%)	160,604 (99.84%)	741,764 (99.24%)
not, retried as stated: proved	292 of 837	3,583 of 4,571	122 of 252	3,997 of 5,660
normal form smaller / same / larger (%)	86 / 14 / 0	25 / 23 / 52	18 / 53 / 29	51 / 25 / 24
goal unchanged by the normalizer	675,781 (57.7%)	104,383 (11.6%)	29,710 (5.7%)	809,874 (31.2%)
normalization and closing certificates (s)	52.1	224.6	189.3	466.0
inserting the closed holes’ certificates (s)	28.7	1,725.7	944.2	2,698.5
worker time of the other justified holes (h)	60.2	112.2	26.9	199.3

Table 2: The normalizer in the elaboration pass. “Normal form tried first”: egglog on the equality of the normal forms, on a third of the hole’s budget; “bridged”: its certificate connected to the stated sides by the normalizer’s derivations. Normalization time includes building the closing certificates.

Before rw5, measured on and off: over the whole corpus at theory-rewrite granularity (run **enc4**, checking only, 5.44 M holes) the normalizer took the proved holes from 70.7% to 98.3% and the passes from 112 h to 50 h; at rewrite granularity on ten sample proofs, from 85.2% to 99.7% of the holes and from 2 to 6 fully proved proofs, in a quarter of the time. A rewrite-granularity hole is one call of the whole rewriter on a term, and without the normalizer egglog pays for its size.

3.2 The cost of a hole

A hole’s cost is its worker time: the wall time of its processes, every attempt included (Table 4, Figure 1). In QF_UF a hole egglog proves costs 0.21 s on average and hardly any deviates (p99 0.33 s): the fixed cost of a fresh process, the rule program and a short saturation, of which about

		holes	proved	closed by the normalizer	proofs fully proved	pass time
enc4 : theory-rewrite, 7,192 proofs (2026-09-20)	without	5,441,531	70.7%	–	3,523	112 h
	with	5,440,377	98.3%	57.5%	6,775	50 h
rewrite, ten sample proofs (2026-09-22)	without	7,889	85.2%	–	2	2,094 s
	with	7,889	99.7%	38.4%	6	547 s

Table 3: The normalizer on and off, checking only, before **rw5** (notes §36 and §47.2; 60s and 8 GB per hole and 600s per proof, respectively 30s, 3.5 GB and 300s). In **enc4** most of the difference is holes never attempted within the proof’s budget (1.56 M without, 76,000 with).

0.15 s is the floor. QF_LIA’s cost 1.2 s on average (median 0.75 s, p99 10 s; the tail is Dartagnan’s whole-assertion holes) and QF_LRA’s 0.51 s (median 0.27 s). A kept hole costs what its limits let it spend, 28 to 48 s on average and 17 h in all (8% of the worker time). Over all holes, closed ones included, a hole costs 0.20 s (QF_UF, QF_LRA) or 0.50 s (QF_LIA); the 2,593,653 holes take 216.8 h of worker time in 42.9 h of pass time, a median pass of 7.6 s a proof (Table 6).

	holes	number	%	mean	median	p90	p99	max	hours
QF_UF	closed by the normalizer	155,455	13.27	0.5 ms	0.3 ms	0.3 ms	3.3 ms	49 ms	0.0
	justified through egglog	1,015,831	86.69	213 ms	222 ms	228 ms	326 ms	47.0 s	60.2
	kept	545	0.05	27.8 s	30.0 s	30.2 s	60.2 s	60.2 s	4.2
	all	1,171,831	100.00	198 ms	215 ms	227 ms	316 ms	60.2 s	64.4
QF_LIA	closed by the normalizer	549,155	61.05	3.6 ms	2.4 ms	6.4 ms	21 ms	375 ms	0.5
	justified through egglog	349,365	38.84	1.2 s	750 ms	1.3 s	10.1 s	79.9 s	112.2
	kept	978	0.11	42.3 s	60.0 s	62.6 s	72.9 s	75.3 s	11.5
	all	899,498	100.00	497 ms	3.4 ms	944 ms	5.2 s	79.9 s	124.2
QF_LRA	closed by the normalizer	331,745	63.51	3.4 ms	1.6 ms	5.6 ms	23 ms	2.1 s	0.3
	justified through egglog	190,436	36.46	509 ms	270 ms	892 ms	1.8 s	62.4 s	26.9
	kept	130	0.02	48.1 s	60.0 s	60.2 s	70.6 s	74.2 s	1.7
	all	522,311	100.00	200 ms	3.6 ms	645 ms	1.5 s	74.2 s	29.0

Table 4: Cost per hole. For a hole egglog proves or that is kept, the worker time; for a hole the normalizer closes, its share of the proof’s normalization plus the check of its certificate in the parent. A kept hole’s time is its last attempt’s kill time or its phases, whichever is larger.

3.3 Elaboration against checking

Per hole the split is exact (Table 5). For the 1.56 M holes justified through egglog, the saturation that decides the goal, which is what a checking-only pass runs, is 78% of the worker time; reconstructing the certificate from the saturated e-graph is 11%, nearly all of it the certificate search; inserting and checking its steps 1%; starting the process 10%. The median hole spends 168 ms in egglog and 34 ms in reconstruction and insertion, and for 0.2% of the holes the reconstruction costs more than egglog. What the reconstruction loses costs more: 868 holes were kept in the reconstruction after egglog had proved their goal or a descent pair of it (killed in one of its phases, no certificate, or the certificate rejected), 12 h of worker time.

Per proof it is an estimate: the pass less its holes’ reconstruction phases spread over the eight workers and less the insertion comes to 37.8 h for a checking-only pass, against 42.9 h for elaboration, which therefore adds 12% (Table 6). The result is cheap to check: the 6,585 proofs with every hole justified re-check in 1.0 h in all (median 0.14 s), against the 25.3 h their elaboration took, with twice the steps of the hoisted proof.

3.4 Against cvc5’s own expansion

Table 7, Figure 2. **Producing.** At **dsl-rewrite** cvc5 takes 15% more time (6.1 h against 5.3 h summed, median 0.68 s against 0.53 s), loses 22 proofs to its 60 s limit, and prints 7% more text.

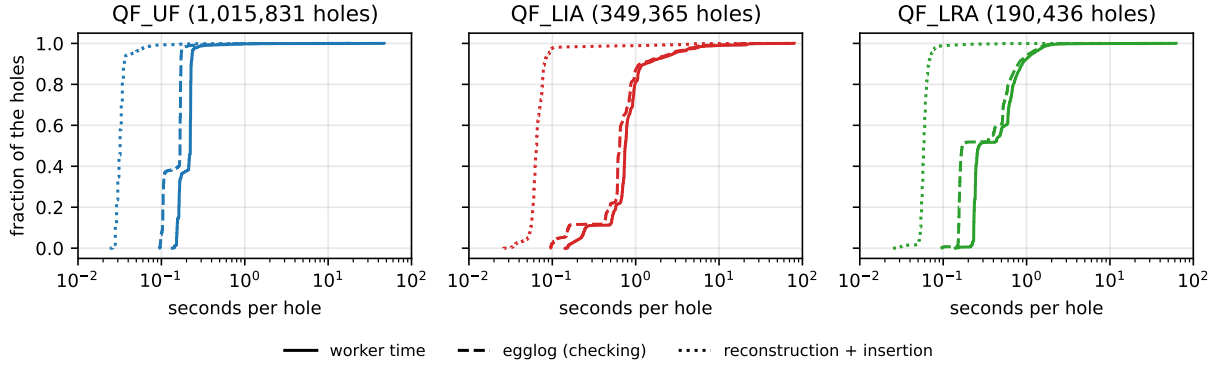


Figure 1: The holes egglog proved: worker time per hole, its egglog phases (what a checking-only pass runs), and its reconstruction phases with the insertion (what elaboration adds). The steps are the per-logic floors: a process, the rule program, the first rounds.

justified holes through egglog, hours	QF_UF	QF_LIA	QF_LRA	all
egglog: saturation, the goal decided	42.3 (70%)	92.0 (81%)	22.0 (81%)	156.3 (78%)
reconstruction	10.0 (16%)	9.8 (9%)	3.1 (12%)	22.9 (11%)
serializing the e-graph	0.4 (1%)	0.7 (1%)	0.2 (1%)	1.3 (1%)
indexing it	0.0 (0%)	0.1 (0%)	0.0 (0%)	0.2 (0%)
searching for the certificate	9.5 (16%)	8.7 (8%)	3.0 (11%)	21.2 (11%)
emitting the steps	0.0 (0%)	0.3 (0%)	0.0 (0%)	0.3 (0%)
inserting and checking the steps	0.2 (0%)	0.9 (1%)	0.1 (0%)	1.3 (1%)
process start and input parsing	7.9 (13%)	10.4 (9%)	1.8 (7%)	20.1 (10%)
per hole, median: egglog / elaboration	167 ms / 32 ms	644 ms / 64 ms	170 ms / 58 ms	168 ms / 34 ms
holes whose elaboration costs more than egglog (%)	0.0	0.4	0.2	0.2
kept after egglog proved the goal: holes, worker time	266, 2.3 h	488, 8.1 h	114, 1.6 h	868, 12.1 h

Table 5: Worker time of the holes egglog proved and the reconstruction justified, by phase. “Elaboration” per hole: reconstruction and insertion. The last row is the kept holes that egglog had proved.

	QF_UF	QF_LIA	QF_LRA	all
proofs whose pass finished	4,118	2,523	518	7,159
elaboration pass (h)	15.9	21.4	5.6	42.9
normalizer	0.0	0.1	0.1	0.1
holes in the workers	14.5	17.8	5.0	37.3
inserting and checking the certificates	0.2	1.4	0.4	2.0
the rest: parsing, the other steps, printing	1.2	2.1	0.2	3.5
checking-only pass, estimated (h)	14.4	18.6	4.8	37.8
elaboration’s share of the pass (%)	9.5	12.9	13.9	11.8
worker time (h)	64.4	123.7	28.7	216.8
utilization of the 8 workers (%)	55.6	86.8	72.2	72.7
pass time per proof, median / p90	9.2 s / 37.1 s	1.5 s / 66.4 s	4.6 s / 125.8 s	7.6 s / 54.9 s
task memory, p99 / max (GB)	3.5 / 7.9	15.4 / 37.8	6.1 / 9.1	6.1 / 37.8
proofs with every hole justified, re-checked	3,718	2,420	447	6,585
elaboration pass, median / sum	8.6 s / 9.7 h	1.3 s / 12.8 h	2.3 s / 2.8 h	6.8 s / 25.3 h
re-check of the elaborated proof, median / sum	272 ms / 0.6 h	15 ms / 0.3 h	52 ms / 0.1 h	138 ms / 1.0 h
steps: hoisted proof / elaborated, median	27,134 / 32,125	126 / 520	468 / 4,138	6,254 / 12,551

Table 6: The elaboration pass per proof. “Holes in the workers” is the wall time of the hole phase; the worker time is what the eight workers spent in it. The checking-only pass is estimated, as described in the text.

Its expansion of every rewrite thus costs it about 0.8 h, around a millisecond a hole, against 0.3 s of worker time a hole in Carcara. **Outcome.** 5,245 proofs re-check `valid` against 4,679; 4,669 in both, 10 only in `rw5`, and 576 only in `dsl1`, 567 of which have a hole `rw5` kept or did not reach. In `QF_UF` both are capped by the `THEORY_LEMMA` holes of 1,789 proofs, which neither attempts. **Time.** On the 4,669 proofs valid in both, a checked proof takes 5.2 s with `rw5` against 0.42 s with `dsl1` (median), 22.5 h against 3.1 h summed; a checking-only pass would not close the gap (Figure 2): the cost is egglog’s, not the reconstruction’s. **The proofs.** `rw5`’s elaborated proofs have 1.4 times `dsl1`’s steps (98.5 M against 68.9 M; 3.2 M `rare_rewrite` steps against 2.4 M, fewer in `QF_LRA`, where the normalizer’s `poly_simp` and `aci_simp` steps stand in) and are checked in the same time (0.5 h each, medians 69 and 58 ms).

	QF_UF	QF_LIA	QF_LRA	all
benchmarks	4,316	2,541	524	7,381
complete proof in 60s: rewrite / dsl-rewrite	4,316 / 4,316	2,539 / 2,531	524 / 510	7,379 / 7,357
cvc5 time, median: rewrite / dsl-rewrite	810 ms / 982 ms	48 ms / 88 ms	198 ms / 388 ms	526 ms / 681 ms
cvc5 time, summed (h): rewrite / dsl-rewrite	3.1 / 3.4	1.4 / 1.8	0.8 / 1.0	5.3 / 6.1
proof text (GB): rewrite / dsl-rewrite	22.0 / 22.3	3.2 / 4.2	1.4 / 2.1	26.7 / 28.6
re-checked <code>valid</code> : <code>rw5</code> / <code>dsl1</code>	1,828 / 2,230	2,404 / 2,510	447 / 505	4,679 / 5,245
both / <code>rw5</code> only / <code>dsl1</code> only	1,828 / 0 / 402	2,402 / 2 / 108	439 / 8 / 66	4,669 / 10 / 576
valid in both, time to a checked proof, median: <code>rw5</code> / <code>dsl1</code>	6.8 s / 617 ms	1.6 s / 90 ms	2.4 s / 185 ms	5.2 s / 415 ms
summed (h): <code>rw5</code> / <code>dsl1</code>	5.0 / 0.7	14.3 / 1.6	3.2 / 0.8	22.5 / 3.1
final proof’s check, median: <code>rw5</code> / <code>dsl1</code>	124 ms / 95 ms	15 ms / 16 ms	50 ms / 29 ms	69 ms / 58 ms
final proof’s check, summed (h): <code>rw5</code> / <code>dsl1</code>	0.1 / 0.1	0.3 / 0.3	0.1 / 0.1	0.5 / 0.5
steps, median: <code>rw5</code> elaborated / <code>dsl1</code>	12,365 / 9,475	502 / 274	3,889 / 1,144	3,918 / 2,372
steps, summed (M): <code>rw5</code> elaborated / <code>dsl1</code>	37.7 / 33.2	40.6 / 24.4	20.3 / 11.3	98.5 / 68.9
<code>rare_rewrite</code> steps, summed (M): <code>rw5</code> / <code>dsl1</code>	1.53 / 0.62	1.54 / 1.21	0.17 / 0.60	3.24 / 2.42

Table 7: `rw5` against `dsl1`, the same benchmarks and `cvc5`; `cvc5` rows over the proofs complete at both granularities. A checked proof’s time: `cvc5`, hoist, elaboration and re-check for `rw5`; `cvc5` and check for `dsl1`.

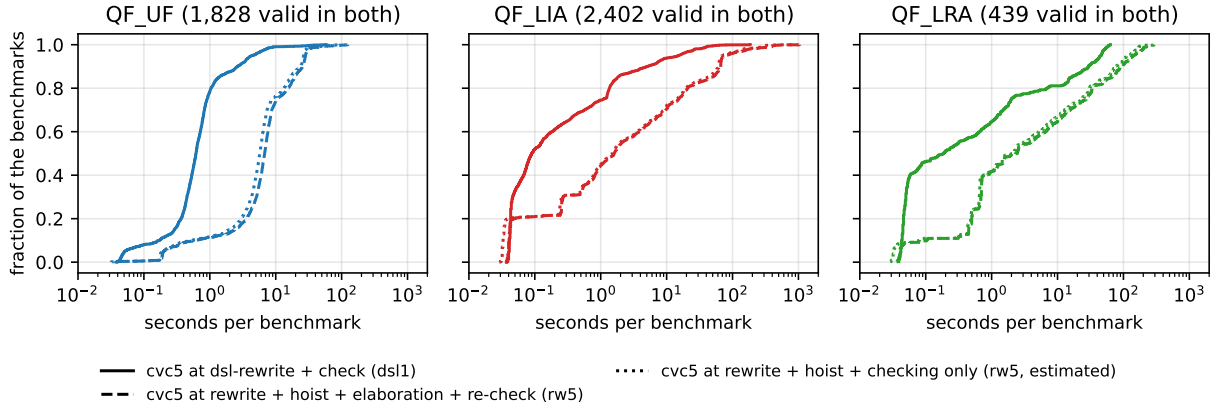


Figure 2: Time to a checked proof on the proofs valid in both arms.

4 In short

1. **Normalizer:** closes 40.0% of the holes at 3 ms each and bridges 99.24% of the normal forms it hands egglog; every kept hole is among the goals it changed. Measured on and off, it is 71% against 98% of the holes proved (whole corpus, theory-rewrite) and 85% against 99.7% (ten proofs, rewrite), in half to a quarter of the time.
2. **Cost of a hole:** 0.20 s of worker time in `QF_UF` and `QF_LRA`, 0.50 s in `QF_LIA`, closed holes included; a hole egglog proves has a floor of about 0.15 s, the process and the rule program.
3. **Elaboration against checking:** 11% of the worker time of a justified hole, 12% of the pass;

its losses are 868 holes and 12 h. The elaborated proof re-checks 25 times faster than it is produced.

4. **Against cvc5:** its own expansion costs it about a millisecond a hole against Carcara's 0.3 s; 576 proofs are valid only with it and 10 only with **rw5**, and its checked proofs come in a seventh of the time. The proofs **rw5** elaborates are 1.4 times larger and check as fast.